

Karen AI Chat System - Developer Guide

Overview

This guide provides technical documentation for developers working with the refactored Karen AI chat system. It covers how to make changes to the chat system, add new providers or models, modify memory behavior, debug issues, and best practices for the new architecture.

Understanding the Architecture

Before making changes to the system, it's important to understand the key components and their relationships:

Core Components

1. **ChatOrchestrator**: The central coordinator and single source of truth for the chat response lifecycle.
2. **FallbackRouter**: Centralizes all fallback decisions under ChatOrchestrator control.
3. **Route Handler**: Thin ingress layer that handles HTTP-specific concerns and delegates to ChatOrchestrator.
4. **Memory Operations**: Handle memory recall and writeback under ChatOrchestrator control.

Request Flow

1. HTTP request arrives at the route handler
2. Route handler creates a `ChatRequest` object
3. Route handler delegates to `ChatOrchestrator.process_message()`
4. ChatOrchestrator executes the processing pipeline
5. ChatOrchestrator returns a `ChatResponse` object
6. Route handler formats the response and returns it to the client

PROF

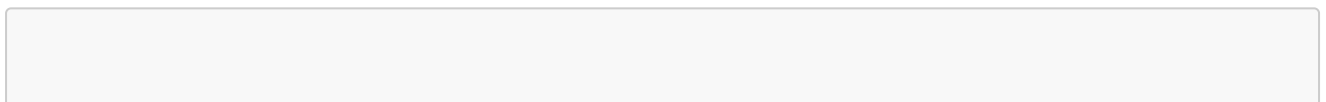
Making Changes to the Chat System

Adding New Processing Steps

To add a new processing step to the chat pipeline:

1. **Identify the appropriate location** in the `_process_message_internal` method
2. **Implement the new step** with proper error handling
3. **Add logging** for monitoring and debugging
4. **Update the processing context** if needed
5. **Add tests** for the new functionality

Example: Adding a Content Moderation Step



```

async def _process_message_internal(
    self,
    request: ChatRequest,
    context: ProcessingContext
) -> ProcessingResult:
    start_time = time.time()
    parsed_message = None
    embeddings = None
    retrieved_context = None
    used_fallback = False
    extracted_instructions = []

    if context.cancel_event.is_set():
        raise asyncio.CancelledError()

    try:
        # Step 1: Parse message with spaCy
        # ... existing code ...

        # Step 2: Extract and process instructions
        # ... existing code ...

        # Step 3: Generate embeddings with DistilBERT
        # ... existing code ...

        # NEW STEP: Content Moderation
        try:
            moderation_result = await
self._moderate_content(request.message, parsed_message)
            if moderation_result["flagged"]:
                logger.warning(
                    "Content flagged by moderation system",
                    extra={
                        "correlation_id": context.correlation_id,
                        "user_id": request.user_id,
                        "categories": moderation_result["categories"],
                        "score": moderation_result["score"]
                    }
                )

                # Decide how to handle flagged content
                if self._should_block_content(moderation_result):
                    return ProcessingResult(
                        success=False,
                        error="Content policy violation",
                        error_type=ErrorType.CONTENT_POLICY_VIOLATION,
                        processing_time=time.time() - start_time,
                        correlation_id=context.correlation_id
                    )
        except Exception as e:
            logger.warning(f"Content moderation failed: {e}")
            # Don't fail the entire request for moderation errors

```

```
# Step 4: Extract and store memories (if memory processor
available)
# ... continue with existing code ...
```

Modifying Existing Processing Steps

To modify an existing processing step:

1. **Understand the current implementation** and its dependencies
2. **Make minimal changes** to maintain system stability
3. **Preserve backward compatibility** when possible
4. **Update tests** to reflect the changes
5. **Document the changes** in the code comments

Example: Enhancing Memory Context Retrieval

```
async def _retrieve_context(
    self,
    embeddings: List[float],
    parsed_message: ParsedMessage,
    user_id: str,
    conversation_id: str
) -> Dict[str, Any]:
    """Retrieve relevant context for the message using
    MemoryProcessor."""
    if not self.memory_processor:
        # Fallback context when memory processor is not available
        return {
            "memories": [],
            "conversation_history": [],
            "user_preferences": {},
            "entities": [{"text": ent[0], "label": ent[1]} for ent in
parsed_message.entities],
            "embedding_similarity_threshold": 0.7,
            "context_summary": "Memory processor not available"
        }

    try:
        # Use MemoryProcessor to get relevant context
        memory_context = await
self.memory_processor.get_relevant_context(
            embeddings,
            parsed_message,
            user_id,
            conversation_id
        )

        # ENHANCEMENT: Add personalized context based on user profile
```

```

user_profile = await self._get_user_profile(user_id)
personalized_context = await self._enhance_context_with_profile(
    memory_context, user_profile
)

# Convert MemoryContext to dictionary format
context = {
    "memories": [
        {
            "id": mem.id,
            "content": mem.content,
            "type": mem.memory_type.value,
            "similarity_score": mem.similarity_score,
            "recency_score": mem.recency_score,
            "combined_score": mem.combined_score,
            "created_at": mem.created_at.isoformat(),
            "metadata": mem.metadata
        }
        for mem in personalized_context.memories
    ],
    "entities": personalized_context.entities,
    "preferences": personalized_context.preferences,
    "facts": personalized_context.facts,
    "relationships": personalized_context.relationships,
    "context_summary": personalized_context.context_summary,
    "retrieval_time": personalized_context.retrieval_time,
    "total_memories_considered":
personalized_context.total_memories_considered,
    "embedding_similarity_threshold":
self.memory_processor.similarity_threshold,
    "user_profile": {
        "preferences": user_profile.get("preferences", {}),
        "personalization_level":
user_profile.get("personalization_level", "standard")
    }
}

return context

```

PROF

Adding New Endpoints

To add new endpoints to the chat system:

1. **Define the request/response models** using Pydantic
2. **Implement the endpoint logic** in the appropriate route file
3. **Delegate to ChatOrchestrator** when appropriate
4. **Add authentication and authorization** if needed
5. **Add tests** for the new endpoint

Example: Adding a Chat History Endpoint

```

# In api_routes/copilot_routes.py

class ChatHistoryRequest(BaseModel):
    user_id: str = Field(..., min_length=1)
    conversation_id: str = Field(..., min_length=1)
    limit: int = Field(50, ge=1, le=100)
    offset: int = Field(0, ge=0)

class ChatHistoryResponse(BaseModel):
    messages: List[Dict[str, Any]]
    total_count: int
    has_more: bool
    correlation_id: str

@router.post("/history", response_model=ChatHistoryResponse)
async def get_chat_history(
    request: ChatHistoryRequest,
    http_request: Request,
):
    """Get chat history for a user and conversation."""
    correlation_id = get_correlation_id(http_request) or
    f"history_{int(time.time())}"

    # Get user context for authorization
    user_context = await _resolve_user_context(http_request)
    if not user_context or user_context.get("user_id") !=
    request.user_id:
        raise HTTPException(status_code=403, detail="Unauthorized to
        access this conversation")

    try:
        # Get chat orchestrator
        chat_orchestrator = _get_chat_orchestrator()

        # Get chat history
        history = await chat_orchestrator.get_chat_history(
            user_id=request.user_id,
            conversation_id=request.conversation_id,
            limit=request.limit,
            offset=request.offset
        )

        return ChatHistoryResponse(
            messages=history["messages"],
            total_count=history["total_count"],
            has_more=history["has_more"],
            correlation_id=correlation_id
        )

    except Exception as e:
        logger.error(
            "Failed to get chat history: %s",

```

```

        e,
        extra={"correlation_id": correlation_id}
    )

    raise HTTPException(
        status_code=500,
        detail="Failed to retrieve chat history"
    )

```

Adding New Providers or Models

Adding a New LLM Provider

To add a new LLM provider to the system:

1. **Implement the provider interface** in the LLM orchestrator
2. **Add configuration** for the new provider
3. **Update the fallback chain** to include the new provider
4. **Add tests** for the new provider
5. **Update documentation** with provider details

Example: Adding a New LLM Provider

```

# In llm_orchestrator.py

class NewLLMProvider(BaseLLMProvider):
    """Implementation of a new LLM provider."""

    def __init__(self, config: Dict[str, Any]):
        self.api_key = config.get("api_key")
        self.base_url = config.get("base_url",
        "https://api.newllm.com/v1")
        self.model = config.get("model", "newllm-default")

    async def generate(
        self,
        prompt: str,
        max_tokens: int = 1000,
        temperature: float = 0.7,
        **kwargs
    ) -> LLMResponse:
        """Generate a response using the new LLM provider."""
        headers = {
            "Authorization": f"Bearer {self.api_key}",
            "Content-Type": "application/json"
        }

        payload = {
            "model": self.model,

```

```

        "prompt": prompt,
        "max_tokens": max_tokens,
        "temperature": temperature,
        **kwargs
    }

    async with aiohttp.ClientSession() as session:
        async with session.post(
            f"{self.base_url}/completions",
            headers=headers,
            json=payload
        ) as response:
            if response.status == 200:
                data = await response.json()
                return LLMResponse(
                    content=data["choices"][0]["text"],
                    model=self.model,
                    provider="newllm",
                    usage=data.get("usage", {}),
                    metadata={
                        "api_version": data.get("api_version"),
                        "finish_reason": data["choices"]
[0].get("finish_reason")
                    }
                )
            else:
                error_data = await response.json()
                raise LLMPProviderError(
                    f"NewLLM API error: {error_data.get('error',
{}).get('message', 'Unknown error')}",
                    provider="newllm",
                    status_code=response.status
                )

# Register the new provider
def register_newllm_provider():
    """Register the new LLM provider with the orchestrator."""
    from ai_karen_engine.llm_orchestrator import get_orchestrator

    orchestrator = get_orchestrator()
    orchestrator.register_provider("newllm", NewLLMProvider)

```

PROF

Adding a New Model to an Existing Provider

To add a new model to an existing provider:

1. **Update the provider configuration** to include the new model
2. **Update the model registry** with the new model information
3. **Add tests** for the new model
4. **Update documentation** with model details

Example: Adding a New Model to OpenAI Provider

```
# In config/llm_provider_config.py

OPENAI_MODELS = {
    "gpt-4": {
        "max_tokens": 8192,
        "supports_streaming": True,
        "supports_chat": True,
        "cost_per_1k_tokens": 0.03,
        "capabilities": ["code", "reasoning", "creativity"]
    },
    "gpt-4-turbo": {
        "max_tokens": 128000,
        "supports_streaming": True,
        "supports_chat": True,
        "cost_per_1k_tokens": 0.01,
        "capabilities": ["code", "reasoning", "creativity",
"large_context"]
    },
    # NEW MODEL
    "gpt-5": {
        "max_tokens": 32768,
        "supports_streaming": True,
        "supports_chat": True,
        "cost_per_1k_tokens": 0.06,
        "capabilities": ["code", "reasoning", "creativity",
"advanced_reasoning"]
    }
}

# In llm_orchestrator.py

def get_openai_model_info(model_id: str) -> Dict[str, Any]:
    """Get information about an OpenAI model."""
    from ai_karen_engine.config.llm_provider_config import OPENAI_MODELS

    if model_id not in OPENAI_MODELS:
        raise ValueError(f"Unknown OpenAI model: {model_id}")

    return {
        "provider": "openai",
        "model_id": model_id,
        **OPENAI_MODELS[model_id]
    }
```

PROF

Modifying Memory Behavior

Changing Memory Recall Strategy

To change how memories are recalled:

1. **Understand the current recall strategy** in `MemoryProcessor`
2. **Implement the new strategy** with proper error handling
3. **Add configuration options** for the new strategy
4. **Add tests** for the new strategy
5. **Update documentation** with strategy details

Example: Implementing a Hybrid Recall Strategy

```
# In chat/memory_processor.py

class MemoryProcessor:
    def __init__(self, config: Dict[str, Any]):
        self.config = config
        self.recall_strategy = config.get("recall_strategy", "semantic")

    async def get_relevant_context(
        self,
        embeddings: List[float],
        parsed_message: ParsedMessage,
        user_id: str,
        conversation_id: str
    ) -> MemoryContext:
        """Get relevant context using the configured recall strategy."""

        if self.recall_strategy == "semantic":
            return await self._semantic_recall(embeddings,
        parsed_message, user_id, conversation_id)
        elif self.recall_strategy == "keyword":
            return await self._keyword_recall(parsed_message, user_id,
        conversation_id)
        elif self.recall_strategy == "hybrid":
            # NEW STRATEGY: Combine semantic and keyword recall
            semantic_context = await self._semantic_recall(embeddings,
        parsed_message, user_id, conversation_id)
            keyword_context = await self._keyword_recall(parsed_message,
        user_id, conversation_id)

            # Merge the results
            merged_context = self._merge_contexts(semantic_context,
        keyword_context)

            return merged_context
        else:
            logger.warning(f"Unknown recall strategy:
        {self.recall_strategy}, using semantic")
            return await self._semantic_recall(embeddings,
        parsed_message, user_id, conversation_id)

    async def _merge_contexts(
```

```

        self,
        semantic_context: MemoryContext,
        keyword_context: MemoryContext
    ) -> MemoryContext:
        """Merge semantic and keyword contexts."""

        # Combine memories from both strategies
        combined_memories = []

        # Add semantic memories with a boost
        for mem in semantic_context.memories:
            mem.boost_score = mem.combined_score * 1.2 # Boost semantic
matches        combined_memories.append(mem)

        # Add keyword memories that aren't already included
        semantic_ids = {mem.id for mem in semantic_context.memories}
        for mem in keyword_context.memories:
            if mem.id not in semantic_ids:
                mem.boost_score = mem.combined_score * 0.8 # Slightly
reduce keyword matches        combined_memories.append(mem)

        # Sort by boosted score
        combined_memories.sort(key=lambda m: m.boost_score,
reverse=True)

        # Take top memories
        max_memories = self.config.get("max_recall_memories", 10)
        top_memories = combined_memories[:max_memories]

        # Create merged context
        merged_context = MemoryContext(
            memories=top_memories,
            entities=semantic_context.entities +
keyword_context.entities,
            preferences=semantic_context.preferences,
            facts=semantic_context.facts + keyword_context.facts,
            relationships=semantic_context.relationships +
keyword_context.relationships,
            context_summary=f"Hybrid recall: {len(top_memories)}
memories from semantic and keyword search",
            retrieval_time=max(semantic_context.retrieval_time,
keyword_context.retrieval_time),
            total_memories_considered=(
                semantic_context.total_memories_considered +
                keyword_context.total_memories_considered
            )
        )

        return merged_context

```

Changing Memory Writeback Behavior

To change how memories are written back:

1. **Understand the current writeback strategy** in `_orchestrate_post_response_memory_writeback`
2. **Implement the new strategy** with proper error handling
3. **Add configuration options** for the new strategy
4. **Add tests** for the new strategy
5. **Update documentation** with strategy details

Example: Implementing Selective Memory Writeback

```
async def _orchestrate_post_response_memory_writeback(
    self,
    *,
    request: ChatRequest,
    context: ProcessingContext,
    result: ProcessingResult,
) -> Dict[str, Any]:
    """
    Orchestrate post-response memory writeback in a transactional
    manner.

    This method is the SINGLE point of control for all memory writeback
    operations.
    Memory writes only occur AFTER successful response generation,
    ensuring
    transactional integrity - no phantom memory writes from failed
    responses.
    """
    # Transactional guard: Only write back if response generation
    succeeded
    if not result.success or not result.response or not request.user_id:
        logger.debug(
            "Skipping memory writeback for %s: response not successful
or empty",
            context.correlation_id
        )
        return {
            "queued": False,
            "linked_shards": 0,
            "reason": "response_not_successful"
        }

    try:
        from services.memory.internal.memory_writeback import
InteractionType
        from services.memory.unified_memory_service import ContextHit
        from ai_karen_engine.chat.dependencies import get_memory_service
```

```

        memory_service = get_memory_service()
        if memory_service is None or not hasattr(memory_service,
"queue_interaction_writeback"):
            logger.warning(
                "Memory service not available for writeback
orchestration: %s",
                context.correlation_id
            )
            return {"queued": False, "linked_shards": 0, "reason":
"memory_service_unavailable"}

        # NEW FEATURE: Selective memory writeback based on content
quality
        if not self._should_writeback_memory(request, result):
            logger.info(
                "Skipping memory writeback for %s: content did not meet
quality criteria",
                context.correlation_id
            )
            return {
                "queued": False,
                "linked_shards": 0,
                "reason": "content_quality_filter"
            }

        # Step 1: Normalize context hits from retrieved memories
        normalized_hits: List[ContextHit] = []
        raw_memories = result.context.get("memories", []) if
isinstance(result.context, dict) else []
        for item in raw_memories:
            # ... existing normalization logic ...
            normalized_hits.append(
                ContextHit(
                    id=memory_id,
                    text=text,
                    preview=text[:200],
                    score=float(item.get("combined_score") or
item.get("similarity_score") or 0.0),
                    tags=[
                        str(tag)
                        for tag in (item.get("metadata", {}) or
{}.get("tags", []))
                        if isinstance(tag, str)
                    ],
                    meta=item.get("metadata") if
isinstance(item.get("metadata"), dict) else {},
                    importance=int((item.get("metadata", {}) or
{}.get("importance", 5))),
                    decay_tier=str((item.get("metadata", {}) or
{}.get("decay_tier", "short")),
                    created_at=created_at,
                    updated_at=None,

```

```

        user_id=request.user_id,
        org_id=str(context.metadata.get("org_id")) if
context.metadata.get("org_id") else None,
    )
)

# Step 2: Link response to source memory shards
shard_links = []
if normalized_hits and hasattr(memory_service,
"link_response_to_shards"):
    shard_links = await memory_service.link_response_to_shards(
        response_id=context.correlation_id,
        response_content=result.response,
        source_context_hits=normalized_hits,
        user_id=request.user_id,
        org_id=str(context.metadata.get("org_id")) if
context.metadata.get("org_id") else None,
        correlation_id=context.correlation_id,
    )

# Step 3: Queue interaction writeback (the actual memory
persistence)
writeback_id = await memory_service.queue_interaction_writeback(
    content=result.response,
    interaction_type=InteractionType.COPILOT_RESPONSE,
    user_id=request.user_id,
    org_id=str(context.metadata.get("org_id")) if
context.metadata.get("org_id") else None,
    session_id=request.session_id,
    source_shards=shard_links,
    tags=["chat", "response"],
    importance=7, # NEW: Dynamic importance based on content
    metadata={
        "conversation_id": request.conversation_id,
        "user_message": request.message[:1000],
        "llm": result.llm_metadata or {},
        "surface": "chat_orchestrator",
        "orchestrated_by":
"ChatOrchestrator._orchestrate_post_response_memory_writeback",
        # NEW: Add content quality metrics
        "content_quality":
self._calculate_content_quality(request, result),
    },
    correlation_id=context.correlation_id,
)

logger.info(
    "Successfully orchestrated memory writeback for %s:
queued=%s, linked_shards=%d",
    context.correlation_id,
    bool(writeback_id),
    len(shard_links)
)

```

```

        return {
            "queued": bool(writeback_id),
            "linked_shards": len(shard_links),
            "writeback_id": str(writeback_id) if writeback_id else None,
            "normalized_hits": len(normalized_hits)
        }
    except Exception as exc:
        logger.warning(
            "Failed to orchestrate memory writeback for %s: %s",
            context.correlation_id,
            exc,
        )
        return {
            "queued": False,
            "linked_shards": 0,
            "error": str(exc),
            "reason": "writeback_exception"
        }

def _should_writeback_memory(self, request: ChatRequest, result:
ProcessingResult) -> bool:
    """Determine if the interaction should be written to memory."""

    # Skip writeback for certain types of interactions
    if request.message.strip().lower() in ["hello", "hi", "hey",
"thanks", "thank you"]:
        return False

    # Check response length
    if len(result.response) < 20: # Too short to be useful
        return False

    # Check for meaningful content
    if not self._has_meaningful_content(result.response):
        return False

    # Check for sensitive content
    if self._contains_sensitive_content(request.message,
result.response):
        return False

    # All checks passed, proceed with writeback
    return True

def _calculate_content_quality(self, request: ChatRequest, result:
ProcessingResult) -> Dict[str, float]:
    """Calculate content quality metrics."""

    # Length score (0-1)
    length_score = min(1.0, len(result.response) / 500.0)

    # Information density score (0-1)

```

```

        info_density = self._calculate_information_density(result.response)

        # Relevance score (0-1)
        relevance_score = self._calculate_relevance(request.message,
result.response)

        # Overall quality score (0-1)
        overall_quality = (length_score + info_density + relevance_score) /
3.0

    return {
        "length_score": length_score,
        "information_density": info_density,
        "relevance_score": relevance_score,
        "overall_quality": overall_quality
    }

```

Debugging Issues

Common Issues and Solutions

1. ChatOrchestrator Not Available

Symptoms: Error message "Chat service unavailable" when making requests.

Causes:

- ChatOrchestrator failed to initialize
- Dependencies not available
- Configuration errors

Debugging Steps:

1. Check logs for initialization errors
2. Verify all dependencies are running
3. Check configuration files
4. Test dependency connections

```

# Test ChatOrchestrator initialization
async def test_chat_orchestrator():
    try:
        from ai_karen_engine.chat.factory import get_chat_orchestrator
        orchestrator = get_chat_orchestrator()
        logger.info("ChatOrchestrator initialized successfully")
        return True
    except Exception as e:
        logger.error(f"Failed to initialize ChatOrchestrator: {e}")
        return False

```

2. Memory Writeback Failures

Symptoms: Memory writeback errors in logs, memories not being persisted.

Causes:

- Memory service not available
- Database connection issues
- Invalid data being written

Debugging Steps:

1. Check memory service status
2. Verify database connections
3. Check data being written for validity
4. Review memory service logs

```
# Test memory service connection
async def test_memory_service():
    try:
        from ai_karen_engine.chat.dependencies import get_memory_service
        memory_service = get_memory_service()

        if memory_service is None:
            logger.error("Memory service is None")
            return False

        # Test basic operation
        test_result = await memory_service.queue_interaction_writeback(
            content="Test message",
            interaction_type="test",
            user_id="test_user",
            org_id="test_org",
            session_id="test_session",
            source_shards=[],
            tags=["test"],
            importance=1,
            metadata={"test": True},
            correlation_id="test_correlation_id"
        )

        logger.info(f"Memory service test result: {test_result}")
        return True
    except Exception as e:
        logger.error(f"Memory service test failed: {e}")
        return False
```

PROF

3. LLM Provider Failures

Symptoms: Errors when generating AI responses, fallback mechanisms being triggered.

Causes:

- API keys invalid or expired
- Provider service down
- Rate limiting
- Invalid requests

Debugging Steps:

1. Check provider API keys
2. Verify provider service status
3. Check for rate limiting
4. Review request formatting

```
# Test LLM provider connection
async def test_llm_provider():
    try:
        from ai_karen_engine.llm_orchestrator import get_orchestrator
        orchestrator = get_orchestrator()

        # Test with a simple prompt
        result = orchestrator.route(
            "Hello, please respond with 'Test successful'",
            skill="conversation",
            return_metadata=True
        )

        if result and result.content:
            logger.info(f"LLM provider test successful: {result.content}")
            return True
        else:
            logger.error("LLM provider test returned empty result")
            return False
    except Exception as e:
        logger.error(f"LLM provider test failed: {e}")
        return False
```

PROF

Debugging Tools

Correlation ID Tracking

All requests include a correlation ID that can be used to trace a request through the system:

```
# Extract correlation ID from request headers
correlation_id = request.headers.get("X-Correlation-Id") or
```

```

str(uuid.uuid4())

# Include correlation ID in all log messages
logger.info("Processing request", extra={"correlation_id":
correlation_id})

# Pass correlation ID to downstream services
response = await downstream_service.call(
    data=request_data,
    headers={"X-Correlation-Id": correlation_id}
)

```

Debug Logging

Enable debug logging for detailed information:

```

import logging

# Set log level to DEBUG
logging.getLogger("ai_karen_engine.chat").setLevel(logging.DEBUG)

# Enable debug logging for specific components
logging.getLogger("ai_karen_engine.chat.chat_orchestrator").setLevel(log
ging.DEBUG)
logging.getLogger("ai_karen_engine.chat.memory_processor").setLevel(logg
ing.DEBUG)

```

Performance Profiling

Profile the performance of different components:

```

import time
import cProfile
import pstats

def profile_chat_request():
    """Profile a chat request to identify performance bottlenecks."""

    # Create a test request
    request = ChatRequest(
        message="Hello, this is a test message for profiling",
        user_id="test_user",
        conversation_id="test_conversation"
    )

    # Profile the request
    profiler = cProfile.Profile()
    profiler.enable()

```

```
# Run the request
orchestrator = get_chat_orchestrator()
response = orchestrator.process_message(request)

profiler.disable()

# Print profiling results
stats = pstats.Stats(profiler)
stats.sort_stats(pstats.SortKey.TIME)
stats.print_stats(10) # Print top 10 time-consuming functions
```

Best Practices for the New Architecture

1. Follow the Single Responsibility Principle

Each component should have a single, well-defined responsibility:

```
# Good: Single responsibility
class MemoryProcessor:
    """Handles memory operations including recall and writeback."""

    async def get_relevant_context(self, embeddings, parsed_message,
user_id, conversation_id):
        """Retrieve relevant context for the message."""
        pass

    async def extract_memories(self, message, parsed_message,
embeddings, user_id, conversation_id):
        """Extract memories from the message."""
        pass

# Bad: Multiple responsibilities
class ChatProcessor:
    """Handles chat processing, memory operations, and LLM
integration."""

    async def process_message(self, request):
        """Process the chat message."""
        pass

    async def get_relevant_context(self, embeddings, parsed_message,
user_id, conversation_id):
        """Retrieve relevant context for the message."""
        pass

    async def generate_response(self, prompt):
        """Generate AI response."""
        pass
```

2. Use Dependency Injection

Use dependency injection to make components testable and maintainable:

```
# Good: Dependency injection
class ChatOrchestrator:
    def __init__(
        self,
        memory_processor: MemoryProcessor,
        llm_orchestrator: LLMOrchestrator,
        fallback_router: FallbackRouter
    ):
        self.memory_processor = memory_processor
        self.llm_orchestrator = llm_orchestrator
        self.fallback_router = fallback_router

# Bad: Hard dependencies
class ChatOrchestrator:
    def __init__(self):
        self.memory_processor = MemoryProcessor() # Hard dependency
        self.llm_orchestrator = LLMOrchestrator() # Hard dependency
        self.fallback_router = FallbackRouter(self) # Hard dependency
```

3. Implement Proper Error Handling

Implement comprehensive error handling with proper logging:

```
# Good: Comprehensive error handling
async def process_message(self, request: ChatRequest) -> ChatResponse:
    try:
        # Process the message
        result = await self._process_with_retry(request, context)

        if result.success:
            return ChatResponse(
                response=result.response,
                correlation_id=context.correlation_id,
                processing_time=result.processing_time,
                used_fallback=result.used_fallback,
                context_used=bool(result.context),
                metadata=result.metadata
            )
        else:
            logger.error(
                "Message processing failed",
                extra={
                    "correlation_id": context.correlation_id,
                    "error": result.error,
                    "error_type": result.error_type.value if
```

```

        result.error_type else "unknown"
    }
)

    return ChatResponse(
        response="I apologize, but I encountered an error
processing your message.",
        correlation_id=context.correlation_id,
        processing_time=result.processing_time,
        used_fallback=True,
        context_used=False,
        metadata={
            "error": result.error,
            "error_type": result.error_type.value if
result.error_type else "unknown"
        }
    )
except Exception as e:
    logger.error(
        "Unexpected error in message processing",
        exc_info=True,
        extra={"correlation_id": context.correlation_id}
    )

    return ChatResponse(
        response="I apologize, but I encountered an unexpected
error. Please try again.",
        correlation_id=context.correlation_id,
        processing_time=time.time() - start_time,
        used_fallback=True,
        context_used=False,
        metadata={
            "error": str(e),
            "error_type": "unknown"
        }
    )

# Bad: Minimal error handling
async def process_message(self, request: ChatRequest) -> ChatResponse:
    # Process the message
    result = await self._process_message(request)

    return ChatResponse(
        response=result.response,
        correlation_id=result.correlation_id,
        processing_time=result.processing_time,
        used_fallback=result.used_fallback,
        context_used=bool(result.context),
        metadata=result.metadata
    )

```

4. Write Comprehensive Tests

Write tests for all components and edge cases:

```
# Good: Comprehensive tests
import pytest
from unittest.mock import AsyncMock, MagicMock
from ai_karen_engine.chat.chat_orchestrator import ChatOrchestrator,
ChatRequest, ProcessingResult

@pytest.mark.asyncio
async def test_process_message_success():
    # Arrange
    orchestrator = ChatOrchestrator()
    request = ChatRequest(
        message="Hello",
        user_id="test_user",
        conversation_id="test_conversation"
    )

    # Mock dependencies
    orchestrator.memory_processor = AsyncMock()
    orchestrator.memory_processor.get_relevant_context.return_value =
MagicMock()

    # Act
    response = await orchestrator.process_message(request)

    # Assert
    assert response.success is True
    assert response.response is not None

    orchestrator.memory_processor.get_relevant_context.assert_called_once()

@pytest.mark.asyncio
async def test_process_message_memory_failure():
    # Arrange
    orchestrator = ChatOrchestrator()
    request = ChatRequest(
        message="Hello",
        user_id="test_user",
        conversation_id="test_conversation"
    )

    # Mock memory processor to raise an exception
    orchestrator.memory_processor = AsyncMock()
    orchestrator.memory_processor.get_relevant_context.side_effect =
Exception("Memory service unavailable")

    # Act
    response = await orchestrator.process_message(request)
```

```

# Assert
assert response.success is False
assert "memory service unavailable" in response.error.lower()

# Bad: No tests or minimal tests
def test_process_message():
    # No arrangement
    orchestrator = ChatOrchestrator()
    request = ChatRequest(message="Hello", user_id="test_user")

    # No assertion
    response = orchestrator.process_message(request)

```

5. Document Your Code

Document all public methods and classes with clear, concise docstrings:

```

# Good: Well-documented code
class ChatOrchestrator:
    """
    Production-ready chat orchestrator with spaCy and DistilBERT
    integration.

    Features:
    - Message processing pipeline with spaCy parsing and DistilBERT
    embeddings
    - Retry logic with exponential backoff for failed processing
    - Comprehensive error handling with graceful degradation
    - Request correlation and context management

    Args:
        memory_processor (Optional[MemoryProcessor]): Processor for
        memory operations
        file_attachment_service (Optional[FileAttachmentService]):
        Service for file attachments
        multimedia_service (Optional[MultimediaService]): Service for
        multimedia processing
        code_execution_service (Optional[CodeExecutionService]): Service
        for code execution
        tool_integration_service (Optional[ToolIntegrationService]):
        Service for tool integration
        instruction_processor (Optional[InstructionProcessor]):
        Processor for instruction handling
        context_integrator (Optional[ContextIntegrator]): Integrator for
        context processing
        retry_config (Optional[RetryConfig]): Configuration for retry
        logic
        timeout_seconds (float): Timeout for processing in seconds
        enable_monitoring (bool): Whether to enable monitoring
    """

```

```

        auth_service (Optional[Any]): Authentication service
    """

    async def process_message(
        self,
        request: ChatRequest
    ) -> Union[ChatResponse, AsyncGenerator[ChatStreamChunk, None]]:
        """
        Process a chat message with full NLP integration and error
        handling.

        Args:
            request (ChatRequest): Chat request containing message and
            metadata

        Returns:
            Union[ChatResponse, AsyncGenerator[ChatStreamChunk, None]]:
            ChatResponse for non-streaming or AsyncGenerator for
            streaming

        Raises:
            ValueError: If request is invalid
            RuntimeError: If processing fails unexpectedly
        """
        pass

# Bad: Undocumented code
class ChatOrchestrator:
    def __init__(self, memory_processor=None,
file_attachment_service=None, multimedia_service=None,
code_execution_service=None, tool_integration_service=None,
instruction_processor=None, context_integrator=None, retry_config=None,
timeout_seconds=30.0, enable_monitoring=True, auth_service=None):
        pass

    async def process_message(self, request):
        pass

```

PROF

6. Use Configuration Management

Use configuration management to make the system flexible and maintainable:

```

# Good: Configuration management
from ai_karen_engine.config.config_manager import get_config

class ChatOrchestrator:
    def __init__(self, config=None):
        self.config = config or get_config("chat_orchestrator")
        self.timeout_seconds = self.config.get("timeout_seconds", 30.0)
        self.max_retries = self.config.get("max_retries", 3)

```

```
        self.enable_monitoring = self.config.get("enable_monitoring",
True)

        # Initialize components based on configuration
        if self.config.get("enable_memory", True):
            self.memory_processor =
MemoryProcessor(self.config.get("memory", {}))
        else:
            self.memory_processor = None

# Bad: Hard-coded configuration
class ChatOrchestrator:
    def __init__(self):
        self.timeout_seconds = 30.0 # Hard-coded
        self.max_retries = 3 # Hard-coded
        self.enable_monitoring = True # Hard-coded
        self.memory_processor = MemoryProcessor({}) # Hard-coded
```

Conclusion

This developer guide provides comprehensive information for working with the refactored Karen AI chat system. By following the guidelines and best practices outlined in this document, developers can effectively make changes to the chat system, add new providers or models, modify memory behavior, debug issues, and maintain the system in a production environment.

Key takeaways:

1. **Understand the architecture** before making changes
2. **Follow the single responsibility principle** for component design
3. **Use dependency injection** for testable and maintainable code
4. **Implement proper error handling** with comprehensive logging
5. **Write comprehensive tests** for all components and edge cases
6. **Document your code** with clear, concise docstrings
7. **Use configuration management** for flexibility and maintainability

By adhering to these principles, developers can ensure that the Karen AI chat system remains robust, maintainable, and extensible for years to come.